FACULTAD DE INGENIERÍA
Programación Orientada a Objetos

PROYECTO INTEGRADOR

Tercer avance

Herencia, polimorfismo y manejo de excepciones

Proyecto: “Almacén de Paquetes”

Grupo 4

Integrantes

Bolaños Fuentes Juan Manuel
Prado Gutierrez Juan Alejandro
Sánchez Navarro Rodrigo
Villa De Jesús Roberto

Repositorio:
github.com/jmbf010907-ui/Proyecto-Integrador-ALMACEN-DE-PAQUETES

Último commit: Tercer Avance parcial

Fecha de elaboración: 9 de noviembre de 2026

Contenido

1. Cambios respecto al segundo avance y observaciones atendidas
2. Funcionalidades
3. Jerarquía de clases y constructores
4. Polimorfismo
5. UML y organización en paquetes
6. Validaciones y excepciones
7. Pruebas y evidencias
8. Participación del equipo y pendientes

1. Cambios respecto al segundo avance y observaciones atendidas

El sistema conserva el dominio del segundo avance (registro, ubicación y entrega de paquetes en un almacén) y lo evoluciona con herencia, polimorfismo, una excepción propia, un menú de consola y la corrección de las observaciones recibidas.

Observaciones de la retroalimentación

Observación	Qué se hizo	Dónde se comprueba
Al registrar con la fecha nula, el anaquel quedaba ocupado.	**GestorAlmacen.registrarPaquete** valida paquete, fecha, código duplicado y anaquel libre, y crea el **Envio** antes de tocar el anaquel. El anaquel se ocupa hasta el final (validar, crear y solo después modificar).	PruebasAlmacen: «Fecha nula: ningún anaquel ocupado y ningún envío» (E10).
Comprobar que un registro rechazado deje libre la ubicación.	Se probó cada causa de rechazo (fecha nula, paquete nulo, código duplicado, almacén lleno) y se verifica que no cambie ningún anaquel ni el historial.	PruebasAlmacen (E10) y caso P5 (E7).
Proteger los cambios de estado del anaquel y rechazar datos nulos.	**Anaquel.liberar()** lanza IllegalStateException si ya está libre; **ocupar** rechaza nulos y anaqueles ocupados; **Paquete.setEstado** solo acepta EN_ALMACEN o ENTREGADO; la entrega valida receptor, fecha y estado antes de modificar.	PruebasAlmacen (E10): bloques «Protección del estado» y «Entregas rechazadas».

Cambios de diseño

Aspecto	Segundo avance	Tercer avance
Tipos de paquete	Una sola clase Paquete.	Paquete (base) con PaqueteEstandar y PaqueteFragil.
Costo de almacenaje	No existía.	Método polimórfico calcularCostoAlmacenaje(int dias).
Almacén lleno	IllegalStateException.	Excepción propia verificada AlmacenLlenoException.
Interacción	Main con una demostración fija.	Menú de consola con operaciones sucesivas y salida.
Paquetes del código	modelo, servicio y app.	modelo, servicio, excepciones y app.
Registro	registrarPaquete(codigo, peso, fecha).	registrarPaquete(Paquete p, LocalDate fecha) throws AlmacenLlenoException.

2. Funcionalidades

El sistema ofrece las siguientes funcionalidades, todas accesibles desde el menú de **app.Main**. Las cuatro principales demostradas con evidencia son: **registrar paquete con asignación de anaquel** y **registrar entrega** (las dos del segundo avance, corregidas) y, como adicionales, **calcular el costo de almacenaje** y **consultar pendientes**. También se muestran la búsqueda y el historial.

#	Funcionalidad	Método responsable	Menú	Evidencia
1	Registrar paquete (estándar o frágil) y asignar anaquel libre	GestorAlmacen.registrarPaquete	1 y 2	E3, E4
2	Registrar la entrega: cierra el envío, pasa el paquete a ENTREGADO y libera el anaquel	GestorAlmacen.registrarEntrega	4	E8
3	Calcular el costo de almacenaje (polimórfico, *nueva*)	GestorAlmacen.calcularCostoAlmacenaje	5 y 8	E6, E9
4	Ver paquetes pendientes (N o más días guardados, *nueva en el menú*)	GestorAlmacen.getPendientes	6	E5
5	Buscar un paquete por código	GestorAlmacen.buscarPaquete	3	E5
6	Ver historial de envíos y estado de los anaqueles	GestorAlmacen.getHistorial / getAnaqueles	7	E7

3. Jerarquía de clases y constructores

Clase base: Paquete. **Subclases:** PaqueteEstandar y PaqueteFragil.

Relación «es un»

Un paquete estándar *es un* paquete y un paquete frágil *es un* paquete: ambos tienen código, peso, estado, remitente, destinatario y anaquel, y el sistema los registra, ubica y entrega de la misma manera. Por eso el almacén, el envío y el anaquel trabajan con referencias de tipo Paquete y aceptan cualquiera de los dos tipos. No se convirtieron en herencia las demás relaciones (Envio, Anaquel, Entrega…), porque no cumplen la relación «es un».

Qué va en la base y qué en las subclases

Clase	Contiene	Diferencia que aporta
Paquete (base)	Atributos privados comunes (código, peso, estado, remitente, destinatario, anaquel), validaciones, getters, setEstado, calcularCostoAlmacenaje (tarifa general) y validarDias.	Evita repetir código en las subclases.
PaqueteEstandar	Constantes DIAS_GRATIS = 3 y TARIFA_DIA = 5.0.	Los primeros 3 días son gratis; después 5 pesos por día.
PaqueteFragil	Constante TARIFA_DIA = 12.0 y atributo privado cargoManejo.	Cargo fijo de manejo más 12 pesos por día desde el primer día.

Constructores, this y super(...)

Cada subclase invoca al constructor de la base con **super(...)** para crear un objeto válido; **this** distingue el atributo del parámetro. Los atributos siguen encapsulados (private) y se leen con getters.

```
public PaqueteFragil(String codigo, double pesoKg, Remitente remitente,
                     Destinatario destinatario, double cargoManejo) {
    super(codigo, pesoKg, remitente, destinatario);   // constructor de la base
    if (Double.isNaN(cargoManejo) || Double.isInfinite(cargoManejo) || cargoManejo < 0) {
        throw new IllegalArgumentException("El cargo de manejo no puede ser negativo.");
    }
    this.cargoManejo = cargoManejo;   // this: atributo; cargoManejo: parámetro
}
```

4. Polimorfismo

El método **calcularCostoAlmacenaje(int dias)** se define en Paquete y se sobrescribe con **@Override** en las dos subclases:

```
// PaqueteEstandar
@Override
public double calcularCostoAlmacenaje(int dias) {
    validarDias(dias);
    int diasCobrados = Math.max(0, dias - DIAS_GRATIS);
    return diasCobrados * TARIFA_DIA;
}

// PaqueteFragil
@Override
public double calcularCostoAlmacenaje(int dias) {
    validarDias(dias);
    return cargoManejo + dias * TARIFA_DIA;
}
```

Demostración en ejecución (opción 8 del menú, **Main.demostrarPolimorfismo**): un arreglo de tipo Paquete guarda un objeto de cada subclase y se llama al mismo método:

```
Paquete[] paquetes = {
    new PaqueteEstandar("DEMO-E", 1.0, r, d),
    new PaqueteFragil("DEMO-F", 1.0, r, d, 20.0)
};
for (Paquete p : paquetes) {
    System.out.printf("Referencia: Paquete | Objeto: %s | Costo: $%.2f%n",
            p.getClass().getSimpleName(), p.calcularCostoAlmacenaje(5));
}
```

Tipo de la referencia	Objeto que se creó	Implementación que se ejecutó	Resultado a 5 días
Paquete	PaqueteEstandar	PaqueteEstandar.calcularCostoAlmacenaje: 5 - 3 = 2 días cobrados x 5 pesos	$10.00
Paquete	PaqueteFragil	PaqueteFragil.calcularCostoAlmacenaje: 20 de manejo + 5 días x 12 pesos	$80.00

Ambas llamadas se escriben igual (p.calcularCostoAlmacenaje(5)) y Java decide en ejecución qué versión correr según el **objeto real**, no según el tipo de la referencia. En **GestorAlmacen.calcularCostoAlmacenaje** el paquete también se trata mediante una referencia de tipo Paquete, de modo que la misma opción del menú cobra distinto para un paquete estándar y para uno frágil (evidencias E6 y E9).

5. UML y organización en paquetes

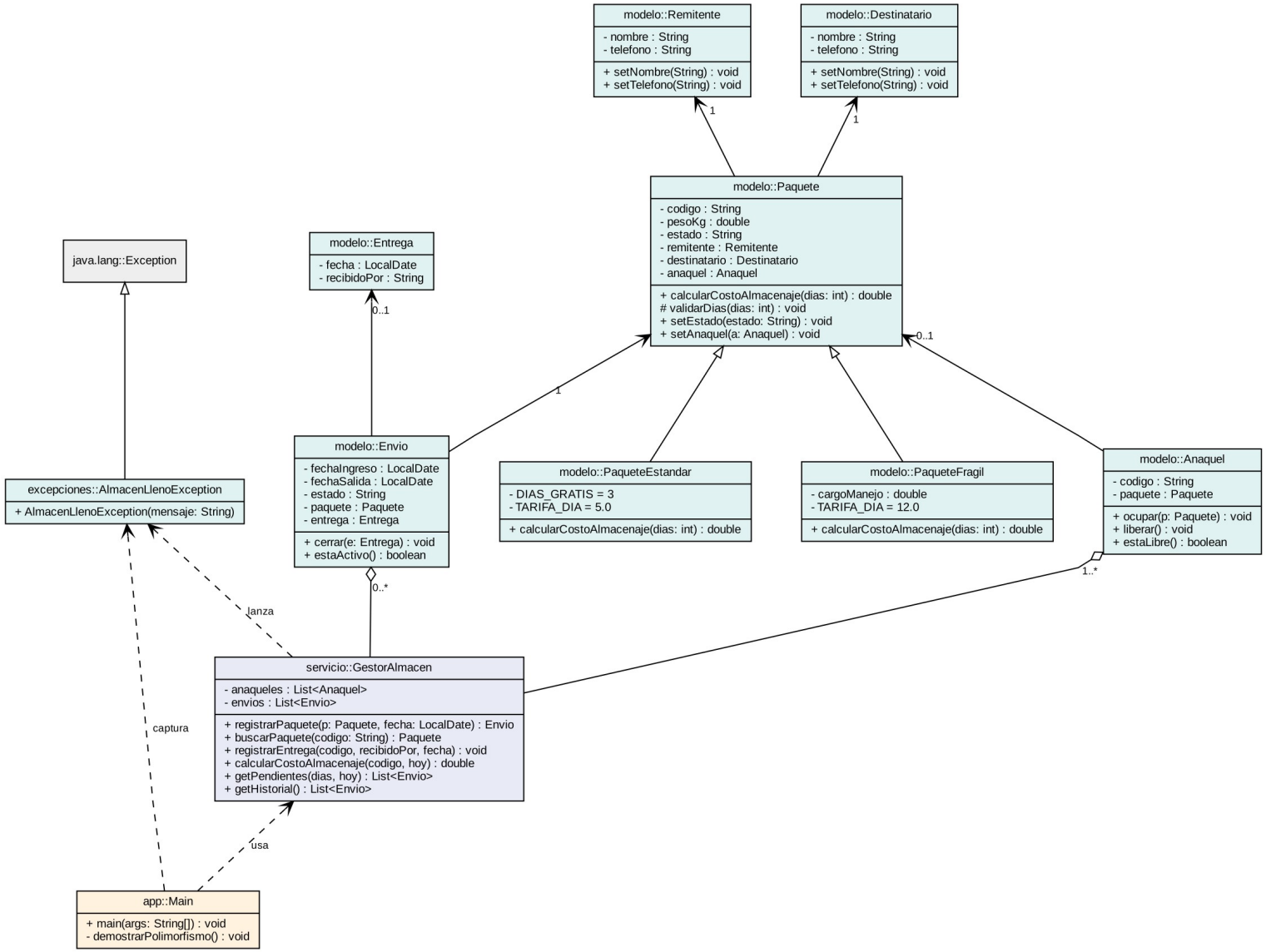


Figura 1. Diagrama de clases actualizado. La flecha con triángulo hueco apunta hacia la clase base (generalización); el rombo hueco indica que GestorAlmacen contiene anaqueles y envíos; la línea punteada es una dependencia. AlmacenLlenoException hereda de Exception. (Imagen: para editarla, rehacer el diagrama en la herramienta de UML del equipo.)

Paquetes del proyecto

Paquete	Clases	Responsabilidad
modelo	Paquete, PaqueteEstandar, PaqueteFragil, Remitente, Destinatario, Anaquel, Envio, Entrega	Datos del dominio, validación de su propia información y comportamiento propio (costo de almacenaje).
servicio	GestorAlmacen	Reglas del almacén: registrar, ubicar, buscar, entregar, consultar y calcular costos.
excepciones	AlmacenLlenoException	Excepción propia del proyecto.
app	Main	Menú de consola; coordina la interacción y captura las excepciones para informar al usuario.

Dependencias entre paquetes: **app** importa servicio, modelo y excepciones; **servicio** importa modelo y excepciones; **modelo** no depende de ningún otro paquete.

6. Validaciones y excepciones

Tipos de excepciones utilizados

Excepción	Cuándo ocurre en el sistema	Dónde se lanza y dónde se captura
NumberFormatException	Texto donde se espera un número (peso, cargo de manejo, días): por ejemplo «abc».	La lanza Double.parseDouble / Integer.parseInt; se captura en Main, **antes** que IllegalArgumentException porque es su subclase.
IllegalArgumentException	Peso 0 o negativo, código o nombre vacío, teléfono que no tenga 10 dígitos, remitente o destinatario nulo, fecha nula, código duplicado, días negativos.	Se lanza en modelo y servicio; se captura en Main con un mensaje para el usuario.
IllegalStateException	Ocupar un anaquel ocupado, liberar uno libre, cerrar un envío ya cerrado.	Se lanza en modelo; se captura en Main (entrega).
AlmacenLlenoException (propia, verificada)	Registrar un paquete cuando no queda ningún anaquel libre.	Se lanza con throw y se declara con throws en GestorAlmacen.registrarPaquete; se captura en Main.
DateTimeParseException	Fecha escrita con formato distinto a AAAA-MM-DD.	La lanza LocalDate.parse; se captura en Main.

Excepción propia

```
public class AlmacenLlenoException extends Exception {
    public AlmacenLlenoException(String mensaje) {
        super(mensaje);
    }
}

// GestorAlmacen
public Envio registrarPaquete(Paquete p, LocalDate fecha) throws AlmacenLlenoException {
    ...
    Anaquel libre = buscarAnaquelLibre();
    if (libre == null) {
        throw new AlmacenLlenoException("No hay anaqueles libres; el paquete " + p.getCodigo() + " no se registró.");
    }
    Envio e = new Envio(p, fecha);   // valida antes de tocar el anaquel
    libre.ocupar(p);                 // los cambios de estado, hasta el final
    p.setAnaquel(libre);
    envios.add(e);
    return e;
}
```

try-catch-finally y recuperación

El registro en **Main** usa try-catch-finally dentro del ciclo del menú: el mensaje de éxito está dentro del **try** (solo se ejecuta si todo salió bien) y «Intento finalizado.» está en el **finally** (aparece tanto en intentos válidos como fallidos). Como el manejo está dentro del ciclo, un fallo no termina la sesión y el usuario puede hacer una nueva operación.

```
try {
    ...
    Remitente r = new Remitente(nomR, telR);
    Destinatario d = new Destinatario(nomD, telD);
    Envio envio = almacen.registrarPaquete(p, fecha);
    System.out.println("Registrado con éxito: " + envio);    // solo si se completó
} catch (NumberFormatException ex) {
    System.out.println("Formato incorrecto: escribe un número, por ejemplo 2.5 (sin letras).");
} catch (DateTimeParseException ex) {
    System.out.println("Fecha incorrecta: usa el formato AAAA-MM-DD, por ejemplo 2026-11-15.");
} catch (IllegalArgumentException ex) {
    System.out.println("Dato inválido: " + ex.getMessage());
} catch (AlmacenLlenoException ex) {
    System.out.println("No se pudo registrar: " + ex.getMessage());
} finally {
    System.out.println("Intento finalizado.");
}
```

Validar antes de modificar

Ninguna operación rechazada deja resultados parciales. En el registro se valida todo y solo al final se ocupa el anaquel, se asigna al paquete y se agrega el envío. En la entrega, primero se crea la Entrega (valida fecha y receptor) y se cierra el Envio (valida estado y fecha, y cambia hasta el final); después se cambia el estado del paquete y se libera el anaquel.

Error de compilación, error lógico y excepción de ejecución

Tipo	Qué es	Ejemplo en este sistema
Error de compilación	El compilador rechaza el código; el programa ni siquiera se genera.	Llamar a registrarPaquete sin capturar ni declarar AlmacenLlenoException (es verificada): no compila.
Error lógico	Compila y se ejecuta, pero el resultado o el estado son incorrectos.	En el segundo avance el anaquel se ocupaba antes de validar la fecha: un registro rechazado dejaba el anaquel ocupado. Se corrigió reordenando las operaciones.
Excepción de ejecución	Un problema que surge al correr el programa; se puede capturar y atender.	NumberFormatException al escribir «abc» como peso; IllegalArgumentException con peso -1; AlmacenLlenoException con el almacén lleno.

7. Pruebas y evidencias

Los casos se reproducen ejecutando java -cp out app.Main (menú, con la secuencia de entradas de las evidencias) y java -cp out PruebasAlmacen (verificaciones automáticas). El almacén de las pruebas tiene 3 anaqueles.

Caso	Estado inicial	Acción	Resultado esperado	Resultado obtenido	Evidencia
P1	Almacén con PQ-001 estándar (ingreso 10/oct/2026).	Opción 8 / opción 5 con PQ-001 a 5 días, mediante referencia de tipo Paquete.	Se ejecuta PaqueteEstandar: 2 días cobrados, $10.00.	Objeto: PaqueteEstandar, costo $10.00.	E6, E9
P2	Almacén con PQ-002 frágil (manejo $20).	Misma llamada con un objeto frágil.	Se ejecuta PaqueteFragil: manejo más 12 por día; resultado distinto.	Objeto: PaqueteFragil, costo $80.00 a 5 días ($56.00 a 3 días).	E6, E9
P3	Menú principal.	Registrar paquete y escribir «abc» como peso.	Se captura NumberFormatExceptio n y se indica cómo corregir.	«Formato incorrecto: escribe un número, por ejemplo 2.5 (sin letras).»	E1
P4	Menú principal.	Registrar paquete con peso -1 (con remitente y destinatario válidos).	Se lanza y captura IllegalArgumentExceptio n; no se acepta el dato.	«Dato inválido: El peso debe ser mayor a 0.»; el paquete no se registra.	E2
P5	3 anaqueles ocupados (A-01 a A-03), 3 envíos.	Registrar PQ-004 con el almacén lleno.	Se informa el motivo con AlmacenLlenoException y se conserva el estado previo.	Antes: 3 envíos, 3 anaqueles ocupados. Después: igual; PQ-004 no existe.	E7
P6	Después de los fallos P3, P4 y P5.	Registrar un paquete válido (y registrar una entrega).	El sistema continúa y ejecuta la nueva solicitud.	PQ-001 registrado en A-01; tras el fallo de P5, la entrega de PQ-001 y el registro de PQ-004 funcionan.	E3, E8
P7	Menú principal.	Un intento válido y uno fallido de registro.	«Intento finalizado.» en ambos; la confirmación de éxito solo en el válido.	E1, E2 y E3: «Intento finalizado.» en todos; «Registrado con éxito» solo en E3.	E1, E2, E3

Evidencias de ejecución

Salida de consola de las ejecuciones reales de la versión entregada (con las entradas escritas por el usuario). Cada evidencia es un fragmento de una misma sesión. Pueden sustituirse o complementarse con capturas de pantalla propias.

E1. P3 y P7 (intento fallido): texto en una entrada numérica

```
Opción: 1
Código del paquete: PQ-001
Peso en kg (ej. 2.5): abc
Formato incorrecto: escribe un número, por ejemplo 2.5 (sin letras).
Intento finalizado.
```

E2. P4 y P7 (intento fallido): peso fuera de las reglas

```
Opción: 1
Código del paquete: PQ-001
Peso en kg (ej. 2.5): -1
Nombre del remitente: Tienda La Esquina
Teléfono del remitente (10 dígitos): 5512345678
Nombre del destinatario: Ana López
Teléfono del destinatario (10 dígitos): 5587654321
Dato inválido: El peso debe ser mayor a 0.
Intento finalizado.
```

E3. Funcionalidad 1, P6 y P7 (intento válido): registro válido de un paquete estándar tras los fallos

```
Opción: 1
Código del paquete: PQ-001
Peso en kg (ej. 2.5): 2.5
Nombre del remitente: Tienda La Esquina
Teléfono del remitente (10 dígitos): 5512345678
Nombre del destinatario: Ana López
Teléfono del destinatario (10 dígitos): 5587654321
Fecha AAAA-MM-DD (Enter = hoy): 2026-10-10
Registrado con éxito: Envío [PQ-001] ingreso 2026-10-10 | ACTIVO
Paquete PQ-001 | 2.5 kg | EN_ALMACEN | A-01 | De: Tienda La Esquina | Para: Ana López | Tipo: Estándar
Intento finalizado.
```

E4. Funcionalidad 1: registro de un paquete frágil

```
Opción: 2
Código del paquete: PQ-002
Peso en kg (ej. 2.5): 1.2
Nombre del remitente: Juan Pérez
Teléfono del remitente (10 dígitos): 5511111111
Nombre del destinatario: María Ruiz
Teléfono del destinatario (10 dígitos): 5522222222
Cargo de manejo en pesos (ej. 20): 20
Fecha AAAA-MM-DD (Enter = hoy): 2026-10-12
Registrado con éxito: Envío [PQ-002] ingreso 2026-10-12 | ACTIVO
Paquete PQ-002 | 1.2 kg | EN_ALMACEN | A-02 | De: Juan Pérez | Para: María Ruiz | Tipo: Frágil (manejo $20.0)
Intento finalizado.
```

E5. Funcionalidades 4 y 5: buscar un paquete y ver pendientes (3 o más días, al 15/oct/2026)

```
Opción: 3
Código a buscar: PQ-002
Paquete PQ-002 | 1.2 kg | EN_ALMACEN | A-02 | De: Juan Pérez | Para: María Ruiz | Tipo: Frágil (manejo $20.0)

Opción: 6
Mínimo de días guardado (ej. 3): 3
Fecha AAAA-MM-DD (Enter = hoy): 2026-10-15
Envío [PQ-001] ingreso 2026-10-10 | ACTIVO
Envío [PQ-002] ingreso 2026-10-12 | ACTIVO
```

E6. Funcionalidad 3: costo de almacenaje de un paquete estándar y de uno frágil (P1 y P2)

```
Opción: 5
Código del paquete: PQ-001
Fecha AAAA-MM-DD (Enter = hoy): 2026-10-15
Tipo de objeto: PaqueteEstandar
Costo de almacenaje: $10.00

Opción: 5
Código del paquete: PQ-002
Fecha AAAA-MM-DD (Enter = hoy): 2026-10-15
Tipo de objeto: PaqueteFragil
Costo de almacenaje: $56.00
```

E7. P5: estado antes, intento con el almacén lleno (excepción propia) y estado después

```
Opción: 7
Historial:
Envío [PQ-001] ingreso 2026-10-10 | ACTIVO
Envío [PQ-002] ingreso 2026-10-12 | ACTIVO
Envío [PQ-003] ingreso 2026-10-13 | ACTIVO
```

```
Anaqueles:
A-01 [ocupado por PQ-001]
A-02 [ocupado por PQ-002]
A-03 [ocupado por PQ-003]

Opción: 1
Código del paquete: PQ-004
Peso en kg (ej. 2.5): 1
Nombre del remitente: Juan Pérez
Teléfono del remitente (10 dígitos): 5511111111
Nombre del destinatario: María Ruiz
Teléfono del destinatario (10 dígitos): 5522222222
Fecha AAAA-MM-DD (Enter = hoy): 2026-10-14
No se pudo registrar: No hay anaqueles libres; el paquete PQ-004 no se registró.
Intento finalizado.

Opción: 7
Historial:
Envío [PQ-001] ingreso 2026-10-10 | ACTIVO
Envío [PQ-002] ingreso 2026-10-12 | ACTIVO
Envío [PQ-003] ingreso 2026-10-13 | ACTIVO
Anaqueles:
A-01 [ocupado por PQ-001]
A-02 [ocupado por PQ-002]
A-03 [ocupado por PQ-003]
```

E8. Funcionalidad 2 y P6: entrega válida y nuevo registro que reutiliza el anaquel liberado

```
Opción: 4
Código del paquete: PQ-001
Nombre de quien recibe: Ana López
Fecha AAAA-MM-DD (Enter = hoy): 2026-10-15
Entrega registrada con éxito. Paquete PQ-001 | 2.5 kg | ENTREGADO | sin anaquel | De: Tienda La Esquina | Para: Ana López | Tipo: Estándar
Intento finalizado.

Opción: 1
Código del paquete: PQ-004
Peso en kg (ej. 2.5): 1
Nombre del remitente: Juan Pérez
Teléfono del remitente (10 dígitos): 5511111111
Nombre del destinatario: María Ruiz
Teléfono del destinatario (10 dígitos): 5522222222
Fecha AAAA-MM-DD (Enter = hoy): 2026-10-15
Registrado con éxito: Envío [PQ-004] ingreso 2026-10-15 | ACTIVO
Paquete PQ-004 | 1.0 kg | EN_ALMACEN | A-01 | De: Juan Pérez | Para: María Ruiz | Tipo: Estándar
Intento finalizado.
```

E9. P1 y P2: polimorfismo con referencias de tipo Paquete (opción 8)

```
Opción: 8
Costo de almacenaje a 5 días, llamando con referencias de tipo Paquete:
Referencia: Paquete | Objeto: PaqueteEstandar | Costo: $10.00
Referencia: Paquete | Objeto: PaqueteFragil | Costo: $80.00
```

E10. Pruebas automáticas (java -cp out PruebasAlmacen): todas las verificaciones en [OK]

```
--- P1 y P2: polimorfismo (referencia Paquete, dos objetos distintos) ---
[OK]   P1 Estándar a 5 días cobra 2 días (3 gratis): 10.0
[OK]   P1 Estándar a 2 días es gratis: 0.0
[OK]   P2 Frágil a 5 días: 20 + 5*12 = 80.0
[OK]   P2 Mismo método, resultado distinto por tipo de objeto
[OK]   Días negativos rechazados
--- P3: formato incorrecto ---
```

```
[OK]    P3 Texto 'abc' en un peso lanza NumberFormatException
--- P4: valores fuera de las reglas ---
[OK]    P4 Peso 0 rechazado
[OK]    P4 Peso negativo rechazado
[OK]    P4 Peso NaN rechazado
[OK]    P4 Código vacío rechazado
[OK]    P4 Remitente nulo rechazado
[OK]    P4 Destinatario nulo rechazado
[OK]    P4 Teléfono de 9 dígitos rechazado
[OK]    P4 Cargo de manejo negativo rechazado
--- Registro correcto ---
[OK]    Registrar: el paquete existe
[OK]    Registrar: estado EN_ALMACEN
[OK]    Registrar: anaquel A-01 asignado
[OK]    Registrar: envío ACTIVO
[OK]    Pendientes: aparece con 3+ días
--- P5: excepción propia (almacén lleno) y estado conservado ---
[OK]    P5 Estado antes: 1 envío y A-01 ocupado
[OK]    P5 Segundo registro lanza AlmacenLlenoException
[OK]    P5 Estado después: sigue 1 envío
[OK]    P5 Estado después: A-01 sigue con PQ-001
[OK]    P5 El paquete rechazado no quedó registrado
--- Rechazos que no deben dejar anaqueles ocupados ---
[OK]    Fecha nula rechazada
[OK]    Fecha nula: ningún anaquel ocupado y ningún envío
[OK]    Paquete nulo rechazado
[OK]    Paquete nulo: ningún anaquel ocupado
[OK]    Código duplicado rechazado
[OK]    Duplicado: solo un anaquel ocupado
--- Protección del estado del anaquel y del paquete ---
[OK]    Liberar un anaquel libre lanza IllegalStateException
[OK]    Ocupar con paquete nulo lanza IllegalArgumentException
[OK]    Ocupar un anaquel ocupado lanza IllegalStateException
[OK]    setEstado(null) rechazado
[OK]    setEstado('XYZ') rechazado
--- Entregas rechazadas no cambian nada ---
[OK]    Entrega con receptor vacío rechazada
[OK]    Entrega con fecha nula rechazada
[OK]    Entrega con fecha anterior al ingreso rechazada
[OK]    Entrega de paquete inexistente rechazada
[OK]    Tras los rechazos: paquete sigue EN_ALMACEN, envío ACTIVO y anaquel ocupado
--- Costo por el gestor (mismo método, dos tipos) ---
[OK]    Costo de PQ-001 (estándar, 5 días) = 10.0
--- Entrega válida ---
[OK]    Entrega: paquete ENTREGADO
[OK]    Entrega: envío CERRADO
[OK]    Entrega: anaquel liberado
[OK]    Entrega: ya no es pendiente
[OK]    Entrega doble rechazada
--- P6: nueva operación válida después de un fallo ---
[OK]    P6 Registro válido después del fallo: queda en A-01 y ACTIVO

Todas las pruebas pasaron.
```

8. Participación del equipo y pendientes

Integrante	Responsabilidad
Bolaños Fuentes Juan Manuel	Main (menú de consola, try-catch-finally y lectura de datos), Remitente, Destinatario, README, integración del repositorio. Portada y secciones 1, 2 y 8 del reporte.
Prado Gutierrez Juan Alejandro	Clases Paquete (base), PaqueteEstandar y PaqueteFragil. Secciones 3 y 4 del reporte.
Sánchez Navarro Rodrigo	Clases Anaquel y AlmacenLlenoException; UML. Secciones 5 y 6 del reporte.
Villa De Jesús Roberto	Clase GestorAlmacen y pruebas (PruebasAlmacen). Sección 7 del reporte y evidencias.

Limitaciones y pendientes

- Los datos se guardan solo en memoria; al cerrar el programa se pierden (la persistencia queda para un avance posterior).
- La asignación automática de anaqueles según dimensiones, peso o categoría y la consolidación de envíos siguen pendientes.
- El almacén tiene un número fijo de anaqueles (3) definido en Main.